

Kokkos in a Nutshell: API Crash Course

Edgar

May 19, 2026

1. Kokkos Fundamentals

The Compilation Model

Kokkos relies on modern CMake to configure its backend hardware.

- **CPU Builds:** Enable `Kokkos_ENABLE_OPENMP=ON`
- **GPU Builds:** Enable `Kokkos_ENABLE_CUDA=ON` and specify architecture (e.g., `-DKokkos_ARCH_AMPERE86=ON`).
- **Compiler Wrapper:** For older CMake versions, `nvcc_wrapper` allows `nvcc` to process standard C++ code.
- **FetchContent:** CMake automatically downloads and links the Kokkos source code directly from GitHub.

The Execution Lifecycle & RAII

Kokkos must boot up and shut down hardware resources cleanly. -

`Kokkos::initialize()`: Boots execution spaces. -

`Kokkos::finalize()`: Safely destroys execution spaces. - **The “Lifecycle Crash”**: `Kokkos::View` memory is reference-counted. If it is destroyed *after* `finalize()`, the program will Segfault.

The Fix: Scoping

Always wrap Kokkos objects in a scoping block { ... }, or use modern C++ RAI:

```
int main(int argc, char* argv[]) {  
    Kokkos::initialize(argc, argv);  
    { // The Scoping Block  
        Kokkos::View<double*> data("d", 100);  
    }  
    Kokkos::finalize();  
    return 0;  
}
```

// Modern RAI alternative

```
Kokkos::ScopeGuard kokkos(argc, argv);
```

2. Data Management

A Kokkos::View is a multidimensional, reference-counted smart pointer to an array of data.

```
Kokkos::View<double**> my_grid("Name", rows, cols);
```

- **Memory Spaces:** Defaults to DefaultExecutionSpace (GPU). Explicitly allocate on CPU using Kokkos::HostSpace.
- **Data Layouts:** - CPUs prefer **Row-Major** (LayoutRight).
 - GPUs prefer **Column-Major** (LayoutLeft).
 - Kokkos chooses the optimal layout automatically!

Data Transfer: The PCIe Bottleneck

Seamlessly accessing GPU memory from the CPU causes severe performance penalties (page faults).

- **Deep Copy:** Move data across the PCIe bus.

```
Kokkos::deep_copy(dest, source);
```

- **Mirror Views:** Creates a host buffer and transfers data.

```
auto h_view = Kokkos::create_mirror_view_and_copy(  
    Kokkos::HostSpace(),  
    d_view  
);
```

3. Parallel Execution & Patterns

The parallel_for Loop

Replaces standard C++ for loops.

```
Kokkos::parallel_for("Label", Iterations,  
    KOKKOS_LAMBDA(const int i) {  
        // Loop body  
    }  
);
```

- **KOKKOS_LAMBDA:** A macro ([=] __device__ __host__) that captures variables by value for GPU compatibility.
- **MDRangePolicy:** Used for tightly nested loops (2D/3D). Allows Kokkos to optimize hardware thread tiling.

Subviews (Function Abstraction)

Kokkos::subview extracts slices without new memory.

```
// Extract a 1D boundary from a 2D grid
```

```
auto halo = Kokkos::subview(grid, Kokkos::ALL, n);
```

- **Use Case:** Extracting a boundary to pass into MPI.
- **Zero-Copy:** Creates a new smart pointer with a customized stride map. Modifying it modifies the original grid.
- **Pro-Tip:** Since strided C++ types are complex, always capture subviews using `auto`.

4. Advanced Performance

Asynchronous Execution (Streams)

To achieve overlap (e.g., compute while MPI sends data), create specific hardware queues:

```
// Create an independent hardware stream
```

```
Kokkos::DefaultExecutionSpace instance;
```

```
// Always remember to synchronize the queue:
```

```
instance.fence();
```

Node-Level Auto-Tuning (KTune)

NuMeriQS KTune replaces static loop heuristics with runtime optimization (Nelder-Mead algorithm).

```
// Drop-in replacement for Kokkos::parallel_for  
KTune::parallel_for("Label", policy, lambda);
```

- **Search Phase:** Runs kernel dozens of times to find maximum memory bandwidth.
- **Caching:** Saves parameters to `ktune_cache.dat`. Subsequent runs bypass search for optimal speed.

Cluster Execution & Deployment

When running on an HPC cluster (like JSC), adapt execution commands into Slurm batch scripts:

Example interactive allocation

```
srun --nodes=2 \  
      --ntasks-per-node=1 \  
      ./build_gpu/halo_ktune
```

- Ensure `-DKokkos_ENABLE_SERIAL=ON` is set.
- Delete `ktune_cache.dat` when switching architectures.